

Lab Program 2

Title: Develop a multi-stage Dockerfile for Container Orchestration

STEP 1: Create a directory and navigate inside it

```
BitterSteel@victus:~$ cd Coursework/devops/lab_programs/  
BitterSteel@victus:~/Coursework/devops/lab_programs$ ls  
lab1  lab2  
BitterSteel@victus:~/Coursework/devops/lab_programs$ cd lab2/
```

STEP 2: Create **package.json** file

\$ npm init -y

This creates basic package package.json, then manually edit the package.json to add fields

- "main"
- "start"
- "build"
- "dependencies"

```
GNU nano 7.2  
{  
  "name": "lab2",  
  "version": "1.0.0",  
  "description": "",  
  "main": "dist/index.js",  
  "scripts": {  
    "start": "node dist/index.js",  
    "build": "mkdir -p dist && cp -r src/* dist/"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC",  
  "dependencies": {  
    "express": "^5.1.0"  
  },  
  "devDependencies": {}  
}
```

STEP 3: Install dependencies

```
$ npm install
```

STEP 4: Create src directory, then create **index.js** file inside src/

```
$ mkdir src
```

```
$ nano src/index.js
```

Write the node application

```
GNU nano 7.2
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req,res) => {
    res.send("This is a multi-stage Docker!");
});

app.listen(port, () => {
    console.log(`server running at port ${port}`);
});
```

STEP 5: Write Multi-stage **Dockerfile**

\$ nano Dockerfile

```
GNU nano 7.2
#Build stage
FROM node:20-alpine AS builder

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

RUN npm run build

#Production stage
FROM node:20-alpine

WORKDIR /app

COPY --from=builder /app/package*.json ./
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules

EXPOSE 3000

CMD ["node", "dist/index.js"]
```

Project Structure

lab2/

node_modules

src/index.js

Dockerfile

package-lock.json

package.json

```
BitterSteel@victus:~/Coursework/devops/lab_programs/lab2$ ls
Dockerfile  node_modules  package.json  package-lock.json  src
```

STEP 6: Build Docker image

\$ docker build -t lab2 .

```
BitterSteel@victus:~/Coursework/devops/lab_programs/lab2$ docker build -t lab2 .
[+] Building 4.6s (14/14) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 396B
=> [internal] load metadata for docker.io/library/node:20-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [builder 1/6] FROM docker.io/library/node:20-alpine@sha256:6178e78b972f79c335df281f4b7674a2
=> [internal] load build context
=> => transferring context: 43.85kB
=> CACHED [builder 2/6] WORKDIR /app
=> CACHED [builder 3/6] COPY package*.json ./
=> CACHED [builder 4/6] RUN npm install
=> [builder 5/6] COPY . .
=> [builder 6/6] RUN npm run build
=> CACHED [stage-1 3/5] COPY --from=builder /app/package*.json ./
=> CACHED [stage-1 4/5] COPY --from=builder /app/dist ./dist
=> CACHED [stage-1 5/5] COPY --from=builder /app/node_modules ./node_modules
=> exporting to image
=> => exporting layers
=> => writing image sha256:11ea7184d7747701e9826868bd4c84f211485d0a6de1477af998b424de77a6ec
=> => naming to docker.io/library/lab2
```

Verify the docker image

\$ docker images

```
BitterSteel@victus:~/Coursework/devops/lab_programs/lab2$ docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
lab2                 latest      11ea7184d774  9 minutes ago  137MB
```

STEP 7: Run the docker container in detached mode, and bind the port

\$ docker run -it -d -p 8080:3000 --name lab2-container lab2

```
BitterSteel@victus:~/Coursework/devops/lab_programs/lab2$ docker run -it -d -p 8080:3000 --name lab2-container lab2
cb96bfed2ce450612dff39ba2ae957299e91c4ee92b13c372f1ef54439dc1857
```

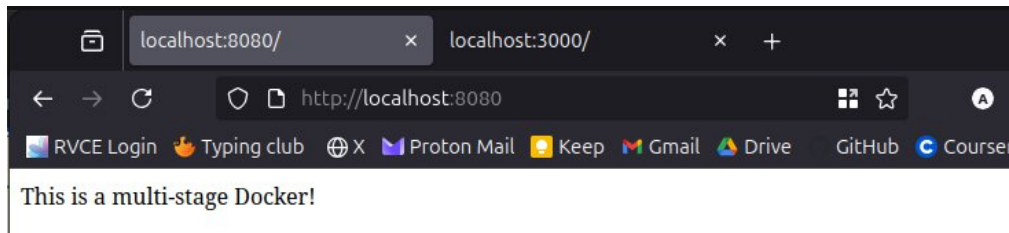
Check if container running

\$ docker ps

```
BitterSteel@victus:~/Coursework/devops/lab_programs/lab2$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
cb96bfed2ce4   lab2     "docker-entrypoint.s..." 6 seconds ago Up 5 seconds  0.0.0.0:8080->3000/tcp, [::]:8080->3000/tcp  lab2-container
```

STEP 8: Access the application in browser

URL - <http://localhost:8080>



STEP 9: Stop the container

\$ docker stop <container-id>

```
BitterSteel@victus:~/Coursework/devops/lab_programs/lab2$ docker stop cb9
cb9
```

STEP 10: Remove the docker container & docker image

\$ docker rm <container-id>

```
BitterSteel@victus:~/Coursework/devops/lab_programs/lab2$ docker rm cb9
cb9
```

\$ docker image rm <image-name>

```
BitterSteel@victus:~/Coursework/devops/lab_programs/lab2$ docker image rm lab2
Untagged: lab2:latest
Deleted: sha256:11ea7184d7747701e9826868bd4c84f211485d0a6de1477af998b424de77a6ec
```